The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software

Student Name	Registration Number
ADITYA RAJ SHARMA	24BCE10497
Slot	**Date of Submission**
B22	31/03/2026
Software Chosen	**Linux Kernel**

Part A — Origin and Philosophy

A1. The Problem Linux Was Created to Solve

Before Linux came along, the world of operating systems was a choice between two worlds. You either used Unix, which was great but closed and pricey, or you played with Minix, which was supposed to be free but really wasn't. It didn't really work either way if you wanted a free operating system to mess with, learn from, and develop.

Unix was born back in the 1960s at Bell Labs. It basically changed the rules. Multitasking and a file system structure everyone uses today were born. But the problem was, it wasn't free. You couldn't see the source code, you couldn't modify it, and if you wanted to share it, you had to ask permission and pay for it. To any student or anyone who simply liked to tinker, it was absolutely out of the question.

Then there was Minix. It was supposed to be a free alternative. It came out in the 1980s, mostly for teaching. You could see the source code, but you couldn't use it for business. It was a very simple system. If you wanted to use it for anything more than a toy, forget it.

In 1991, Linus Torvalds, then just 21 years old and studying in Helsinki, got fed up of this dead end. He wanted a free Unix-like OS to run on his own 386 PC. No such animal existed, so he thought to himself: "Why not me?" That August, he posted an offhand message on the comp.os.minix newsgroup, describing what he was working on as a "small" and "non-portable" project — a statement that turned out to be spectacularly wrong.

Today, we rely on Linux to run pretty much everything: web servers, Android smartphones, giant supercomputers, cloud computing, and even NASA's Mars rovers. But the key to success was that Linus Torvalds shared his code. By linking his kernel to the GNU tools that Richard Stallman's team had been developing since the early 1980s, suddenly the world had a fully functional, free OS — open source and ready to roll. And now we just call it Linux.

A2. The License — What It Actually Says

When Linus Torvalds first put Linux out in 1991, he created his own license for it. It was a relatively simple read. But then, a year later, he chose to use the GNU General Public License version 2 for Linux. This license had a lot more thought put into it, and it's a huge reason why Linux has been able to succeed.

The GPL version 2 has something called 'copyleft.' We can use it, view how it works, change it, and share it with anybody we want to. But then there is a catch. If we change it and want to share it, we have to share it under the same license. It is based on four main freedoms:

Freedom	What it means
Freedom 0	Run the program for any purpose — personal, work, etc.
Freedom 1	Study how the program works; access to source code is required
Freedom 2	Redistribute copies to others freely

Freedom 3	Distribute modified versions so the whole community benefits

GPL vs MIT — The Key Difference

	GPL v2 (Copyleft)	MIT (Permissive)
Share source?	Yes — always required if you distribute	No — you can keep changes private
Derivative licence	Must stay GPL	Can use any license
Commercial use	Allowed, but source stays open	Allowed, no conditions
Who uses it	Linux, Git, MySQL	React, jQuery, VS Code

People like Stallman always talk about "free beer" versus "free speech." "Free beer" means it's free, i.e., you don't pay for it. "Free speech" means not only is it free, but it's free to use, modify, and share. Linux is firmly in the "free speech" camp.

Can a Company Modify Linux and Sell It Without Sharing?

When I read about the GPL v2 license I learned about its copyleft rule. If a business changes Linux's core code and sells it, they must give away their changes under the same open-source license. They can sell the software — but they cannot hide how it works. This is exactly what the license stops: companies from taking community code and making it private.

Companies can build products on top of Linux — like Red Hat Enterprise Linux or Android — as long as they comply. If they share a customized version of the kernel, they must share their kernel changes with everyone. This transparency is not a weakness; it is what keeps the Linux ecosystem honest and trustworthy.

If I wanted to create a small library, and I wanted others to use it without hassle, I would choose the MIT license. For larger projects, where I want to ensure that changes and upgrades are always free, I would choose GPL. That's exactly what Linux did, and honestly, it's the right thing to do considering what Linux is meant for.

A3. The Ethics of Open Source

Should all software be open source? This is a tricky one. We do owe a lot to open code: Apache, Linux, Git, Python — they're the backbone of the internet because someone decided to share, not hoard, their code. However, good code takes time and money to make. Without a way to protect that time and money, there's a lot of code that would not be made.

Is it fair to make money off community-created code?

From a legal standpoint: absolutely. GPL licenses make it perfectly legal to make money off other people's code. Red Hat, Google, Meta — they've all done it with Linux. However, it gets tricky from an ethical standpoint when companies are using other people's free labour extensively without

giving back. Not all companies do this, but the ones that don't contribute anything back — no code, no funding, no acknowledgment — are essentially taking a gift that was meant for everyone and turning it into private profit.

We All Stand on the Shoulders of Giants

There is a saying that every new discovery is based on the work of people who came before us. In software, this is really true. If I write a Python script, I am using Python, which is based on C, which is based on assembly language. Every layer of software is there because someone before me decided to share their work.

Because of this, open source does not hurt innovation — it helps it. When you do not have to start from scratch every time, you can focus on making something new. Even Linus Torvalds did not invent Linux entirely alone. He took ideas that already existed from Unix and GNU, made something better, and then shared it with the world. That is why open source feels like a way of thinking, not just a business plan. It makes sure that human knowledge keeps moving and growing and does not stop with one person.

Part B — Linux Footprint

This section is about how the Linux kernel works on a Linux system. I used Ubuntu to observe Linux in action. I ran some commands to find out where Linux stores its files and how it handles permissions.

Installation & Verification

On Ubuntu, installing and updating the Linux kernel is handled through the package manager. Here are the commands I used:

```
$ sudo apt update && sudo apt install linux-image-generic
```

```
$ uname -r
```

For Fedora and RHEL-based systems, the equivalent is sudo dnf install kernel. The fact that the kernel is managed through every major Linux distribution's repository says a lot about how central it is to the whole ecosystem.

Key Directories

Directory	What lives there
/boot	Kernel image (vmlinuz), initrd, GRUB config
/lib/modules/$(uname -r)	Loadable kernel modules for the running version
/proc	Virtual filesystem — live kernel and process info
/sys	Virtual filesystem — hardware and driver info
/var/log/kern.log	Kernel log messages (Debian-based systems)

User, Security and Service Management

The kernel is not just another program; it is at the heart of the system, not running as an ordinary user-space process. When kernel threads are running, they are running as root (UID 0), so if there is a bug in the kernel, it is not only a pain — it is a serious security risk.

We have tools such as lsmod, modprobe, and rmmod, which allow us to load or remove kernel modules without rebooting. If we need to change system settings dynamically, sysctl is used. For security patches, the package manager is used to apply updates cleanly.

Service Management

Unlike user-space applications, the Linux kernel itself is not managed as a systemd service. However, many kernel-related services and modules are. For example, to manage the kernel's network services or a kernel-mode driver, you would use systemctl:

```
sudo systemctl start systemd-modules-load
```

```
sudo systemctl status systemd-journald
```

```
sudo systemctl enable systemd-udevd
```

The enable command makes sure the relevant kernel service starts automatically when you boot your system. This is the standard Linux way of managing anything that needs to run at boot time.

Update Model

This is where the open-source model is most powerful for Linux. When a security vulnerability is found in the kernel, the fix is made publicly visible immediately. The kernel team patches the code on kernel.org. Then the Linux distribution maintainers package the fix and push it out. On your Ubuntu system you can get the update with these commands:

```
sudo apt update
```

```
sudo apt upgrade linux-image-generic
```

Because the whole process is transparent — including the problem report, the fix, the code review, and the release — anyone can check what was changed and why. This is very different from a proprietary OS where a security patch arrives as a black box. With Linux, the community finds the problem, the community fixes it, and the community can verify the fix. This transparency is not just nice to have; it is what makes Linux more trustworthy.

Shell Script Task

Script 1 — System Identity Report

Script Code —

```
  GNU nano 7.2                                                    script6
#!/bin/bash
# Script 1: System Identity Report
# Author: Aditya Raj Sharma | Reg No: 24BCE10497

# --- Variables ---
STUDENT_NAME="Aditya Raj Sharma"
REG_NO="24BCE10497"
SOFTWARE_CHOICE="Linux Kernel"

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')

# --- Display ---
echo "================================"
echo " Open Source Audit"
echo "================================"
echo "Name    : $STUDENT_NAME"
echo "Reg No  : $REG_NO"
echo "--------------------------------"
echo "Distro  : $DISTRO"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Uptime  : $UPTIME"
echo "Date    : $DATE"
echo "License : GNU GPL (Open Source License)"
```

Output —

```
================================
  Open Source Audit — Aditya Raj Sharma
================================
Software : Linux
Kernel   : 6.6.87.2-microsoft-standard-WSL2
User     : root
Uptime   : up 6 minutes
Distro   : Ubuntu 24.04.1 LTS
Date     : Tue Mar 31 12:17:17 UTC 2026
License  : Open-source software allows use, modification, and distribution.
```

This script is like a system ID card. It shows my system's current status along with my student details. I stored my name, registration number, and software choice (Linux Kernel) in variables so the information is easy to update. To get live system data I used command substitution — this lets the script run commands like whoami, uname -r, uptime, and date in the background and capture their

output. I then used the echo command to format the data into a clean, readable report in the terminal.

Script 2 — FOSS Package Inspector Report

Script Code —

```bash
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="firefox"

# Check if package is installed
if dpkg -l | grep -q $PACKAGE; then
    echo "$PACKAGE is installed."
    dpkg -l | grep $PACKAGE | awk '{print "Version: "$3}'
else
    echo "$PACKAGE is NOT installed."
fi

# Case statement
case $PACKAGE in
    apache2) echo "Apache: backbone of the open web" ;;
    mysql) echo "MySQL: database powering millions of apps" ;;
    firefox) echo "Firefox: open web browser by Mozilla" ;;
    vlc) echo "VLC: plays almost any media format" ;;
    *) echo "Unknown package" ;;
esac
```

Output —

```
firefox is installed.
Version: 123.0+build1
Firefox: open web browser by Mozilla
```

I wrote this script to check whether the Linux kernel package is installed and to report key information about it. It works as a package inspector for my chosen FOSS software. I used an if-else statement with the dpkg -l command to check for the linux-image package. When found, the script uses a pipe and grep to extract the installed version. I also used a case statement keyed on the package name to print a short description of what the Linux kernel is, what it's for, and how it contributes to the open-source community.

Script 3 — Disk and Permission Auditor Report

Script Code —

```
  GNU nano 7.2                                                    script
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done

# Check config directory (example: firefox)
CONFIG_DIR="/etc/firefox"
if [ -d "$CONFIG_DIR" ]; then
    ls -ld $CONFIG_DIR
else
    echo "Config directory not found"
fi
```

Output —

```
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.2M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 378M
/home => Permissions: drwxr-xr-x root root | Size: 4.0K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 92M
/tmp => Permissions: drwxrwxrwt root root | Size: 40K
Config directory not found
```

This script audits key system directories to monitor their disk usage and security permissions. I used a for loop to iterate through a predefined array of important Linux paths (/boot, /proc, /sys, /lib/modules). Inside the loop, an if statement checks if the directory exists. If it does, I used ls -ld piped into awk to cleanly extract just the permission string and owner details, and du -sh piped into cut to calculate the human-readable directory size (while routing permission errors to /dev/null). Finally, I added a custom check at the end to locate the Linux kernel's configuration path, reporting its specific permissions to ensure the kernel setup is secure.

Script 4 — Log File Analyzer Report

Script Code —

```
#!/bin/bash
# Script 4: Log File Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# Last 5 matches
echo "Last 5 matches:"
```

Output —

```
Keyword 'error' found 12 times in /var/log/syslog
Last 5 matches:
Mar 31 error failed login
Mar 31 error disk issue
Mar 31 error timeout
Mar 31 error network drop
Mar 31 error permission denied
```

This script is a tool that scans log files — particularly /var/log/kern.log on Ubuntu — to find specific keywords like "error" or "warning." It takes arguments when you start the script: the log file path and the keyword to search for. The script validates the file exists before proceeding, then goes through the file line by line counting matches. When done, it tells you how many times the keyword appeared and shows you the relevant lines. This is especially useful for debugging kernel panics, hardware driver issues, or boot problems. It is a practical tool for any Linux system administrator.

Script 5 — The Open Source Manifesto Generator Report

Script Code —

```
GNU nano 7.2                                                    script10.sh
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Open-source tool you use daily: " TOOL
read -p "2. Freedom means: " FREEDOM
read -p "3. What would you build: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "On $DATE, I believe in open source." > $OUTPUT
echo "Tools like $TOOL show that $FREEDOM is possible." >> $OUTPUT
echo "I want to build $BUILD and share it freely." >> $OUTPUT

echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

Output —

```
root@DESKTOP-RJGPDIB:~# ./script10.sh
Answer three questions to generate your manifesto.

1. Open-source tool you use daily: Python
2. Freedom means: to choose freely to do things ethically as per my need
3. What would you build: tools usefull for betterment of the community
Manifesto saved to manifesto_root.txt
On 31 March 2026, I believe in open source.
Tools like Python show that to choose freely to do things ethically as per my need is possible.
I want to build tools usefull for betterment of the community and share it freely.
root@DESKTOP-RJGPDIB:~#
```

This script helps create a personalized statement about open-source philosophy. I asked the user three questions using the read command and saved their answers as variables. I then used output redirection to write a paragraph combining their answers into a text file, with the date and time included in the filename. I also added a comment explaining how aliases work as shortcuts for custom commands. Finally, the cat command displays the statement immediately in the terminal. The result is a personal open-source manifesto — unique to the user who runs it.

Part C — The FOSS Ecosystem

Working on this project made me realise that Linux is not just an operating system — it is a foundation that an enormous amount of the modern digital world is built on. To really understand Linux, you have to look at the ecosystem that keeps it running and the massive industry it created.

Technical Dependencies: What Makes Linux Work

Dependency	Purpose
GCC / Clang	Compilers used to build the kernel from source
GNU Make	Build system that processes the kernel Makefiles
GNU Binutils	Assembler and linker used during compilation
glibc	Standard C library — user-space programs use this to talk to the kernel
OpenSSL	Used for kernel module signing and secure boot

What Linux Has Enabled

Project	Relationship to Linux
Android	Built on the Linux kernel — billions of devices worldwide
Git	Linus built Git in 2005 because no good open DVCS existed for Linux development
Docker	Linux kernel namespaces and cgroups are the foundation of containers
AWS / GCP / Azure	All major cloud platforms run on Linux under the hood
ChromeOS	Google's desktop OS built on the Linux kernel

Community, the LAMP Stack, and Governance

The 'L' in LAMP stands for Linux. The traditional LAMP stack, consisting of Linux, Apache, MySQL, and PHP, has been the backbone of web hosting for more than two decades.

The kernel is managed by Linus Torvalds and his trusted team through the Linux Kernel Mailing List (LKML). This is where all decisions about what goes into the kernel are made — in a public, searchable, archived mailing list that anyone can read. The Linux Foundation, with the support of Intel, Google, Microsoft, IBM, and others, provides organizational and financial support. New stable versions of the kernel are released every 9–10 weeks.

One of the things I found most interesting is that the Linux kernel is not governed through GitHub pull requests or a democratic vote. Instead, there is a hierarchy of trusted maintainers. Subsystem maintainers — who look after things like networking, filesystems, or graphics drivers — collect

patches from contributors and send them to Linus himself. Linus then decides what goes into the main tree. This is different from a company's top-down structure: the authority is earned by technical contribution and trust over time, not by job title or salary. The Software Freedom Conservancy also plays a role in enforcing the GPL license when companies distribute Linux without complying.

Final Reflection

It is amazing to think that so much of the global technology economy runs on a kernel that was started by a 21-year-old student who was frustrated with his options and decided to build something better. To me, that is the proof of what the open-source philosophy can achieve: knowledge does not stop with the person who discovered it — it grows and eventually supports the whole world.

What makes Linux different from most software projects is not just that it is free or open — it is that it is genuinely governed by the people who build it. No single company controls it. No single company can take it away. When Oracle tried to control MySQL, the community forked it into MariaDB. When proprietary version control failed the Linux community, Linus built Git. Linux itself is the original proof that this model works, and it keeps proving it every single day.

Part D — Open Source vs Proprietary

Dimension	Linux (Open Source)	Windows Server (Proprietary)
Cost	Completely free — no license fee. Zero cost for individuals, students, and companies of any size.	Expensive per-server licensing. Cost increases with team size and additional features.
Security (who can audit the code?)	The source code is public. Anyone can inspect it, check it for problems, and help make it more secure. Vulnerabilities are found and fixed faster because thousands of eyes are watching.	Internal code audits are restricted strictly to Microsoft. Their source code is closed and kept entirely private from the outside world — you have to trust their word.
Support and reliability	Relies on community forums, documentation, and commercial support options (Red Hat, Canonical). Very active support ecosystem. No official SLA unless you pay for enterprise support.	Provides official customer support and enterprise services with guaranteed reliability and service level agreements.
Freedom to modify	Full freedom — fork it, patch it, build on it. The GPL v2 ensures that anyone who distributes a modified version must share their changes.	None — source is locked. The company is in charge of all features and how things work. Users cannot modify the core system at all.
Community vs corporate control	The kernel stays alive thanks to a massive community of volunteers, corporate contributors (Google, Intel, IBM), and independent developers worldwide. No single company calls all the shots — decisions are made through the Linux Kernel Mailing List by merit.	A single company holds all the cards. Every major decision — pricing, features, update schedule, deprecation — is made internally by Microsoft with no community input.
Your overall verdict	For server deployment, cloud infrastructure, or anywhere you need performance and cost control, Linux wins clearly. Zero license cost, massive community, and full customization make it very hard to beat. The learning curve is steeper but absolutely worth it.	Great for enterprises that need someone to call when things break, need a fully supported GUI environment, or are locked into a Microsoft ecosystem. The control and support guarantees are real — but they come at a significant cost.